

CMU 11-768 《AI Agents》跟学指南

顶尖高校把 Agent Engineering 写进课程体系的第一批样本 —— 12 讲公开课件 + 开源作业仓库，搭 Harness → 建 Eval → 用 RL 训 Agent，全链路可动手

官网 cmu-agents.com

讲师 Fried & Neubig (OpenHands 作者)

Fall 2026 · 15 周 26 讲

12 份 slides 全公开

作业仓库 github.com/cmu-agents

YouTube 全程录像

目录 = 你的疑问清单

1. 这是什么课？凭什么值得跟学？
2. 26 讲四大板块怎么排的？
3. 作业链为什么是 Harness→Eval→Training？
4. 从语言模型到 Agent，关键一步是什么？
5. ReAct 循环长什么样？（附我的实现）
6. 观察爆炸怎么办：截断 + 上下文压缩
7. Skills 渐进披露怎么实现？
8. 程序化工具调用强在哪？
9. 同一个循环怎么换域复用？
10. SFT：把专家轨迹灌进模型
11. RL 训 Agent：奖励从哪来？
12. 安全、框架与跟学路线

Q1

这是什么课？凭什么值得跟学？

11-768 是卡内基梅隆大学 (CMU) 2026 秋季新开的研究生课，讲 **基于 LLM 的 AI Agent：工具使用、规划、记忆、训练与安全**。两位主讲都是业界重量级人物：Daniel Fried 和 Graham Neubig——后者是开源 Agent 框架 **OpenHands** 的作者。每讲还有客座专家（如 Sasha Rush、Karthik Narasimhan）。

推荐人 Xudong Han 的判断是：**Agent Engineering 正在被顶尖高校正式写进课程体系**，而 11-768 的 syllabus 贴近一线实践。它的珍贵之处在于**全套资料公开**：

资源	在哪	状态 (已实测)
12 讲课件 PDF	cmu-agents.com/slides/lecture-01...12.pdf	已下载验证，lecture-01 共 39 页 7.3MB
作业 1 起手仓库	github.com/cmu-agents/assignment-1	已克隆（含公开测试，可离线跑）
作业 2 (Eval)	github.com/cmu-agents/assignment-2	已公开
课堂录像	YouTube 播放列表	随学期陆续上传

✅ **判断：学习类。**不是纯趋势评论——有公开课件 + 可运行的开源作业载体，本指南的实验部分就是拿作业 1 的离线测试真实跑出来的。

Q2

26 讲四大板块怎么排的？

课程顺序本身就是一套方法论宣言：**先会调用（能力）→ 再看落地（领域）→ 然后自己造（训练）→ 最后管住它（安全/框架/交互/搜索）。**



彩蛋：客座讲者全是各域旗手——Computer Use 请 JY Koh, Deep Research 请 Akari Asai (RAG 综述作者)，收官请 Sasha Rush 与 Karthik Narasimhan。

Q3

作业链为什么是 Harness→Eval→Training?

推荐人最推崇的就是作业设计。三次个人作业 + 一个团队研究项目，权重 10% / 15% / 15% / 项目 45% (含 proposal/check-in/poster/report)：

作业	截止	干什么	背后方法论
A1 Harness	Sep 14	在开源 LLM 上搭 coding agent 的 harness (本指南实验的主角)	调用模型只是基础，会搭才是工程
A2 Eval	Oct 1	设计评测框架，度量 agent 的正确性与功能	没测评就没有改进的方向 ([[LLM裁判与自动评估]])
A3 Training	Oct 29	实现 SFT/RL 训练流程，改进自己的 agent	从“调 API”升级到“造模型行为”
研究项目	学期末	团队攻一个开放研究问题 (如并行 agent 的安全与可靠性权衡)	课程明说：欢迎把作业发展成论文

✅ 这个顺序 = 搭起来 → 测得准 → 练得强 → 推出去。与工业界共识一致：先有 harness 与 eval，才谈得上 training——没有测量就没有优化。

Q4

从语言模型到 Agent，关键一步是什么？

Lecture 01 给出了全课最核心的一句话心智模型：**Agent = 模型在循环里 (A model in a loop)**。语言模型本来只会“预测下一个 token”，而**工具调用就是另一种可以被预测的 token 序列**——harness 负责解析、校验、执行它，把结果变成观察 token 喂回去：



🔁 → 模型再推理 → 循环，直到 `send_message` 完成

出处：Lecture 01 "An agent is a model in a loop" (ReAct, Yao et al. 2023)

三个立即可用的认知：①**工具定义**是暴露给模型的类型化接口（名字+描述+JSON Schema），chat template 把它渲染成文本；②**工具调用**是 assistant 消息里的结构化请求，**工具结果**是 `role:"tool"` 消息、靠 `tool_call_id` 与调用挂钩；③Russell & Norvig 的经典定义在这里落地：环境=仓库/网站/工作流，传感器=消息/文件/截图，执行器=工具。

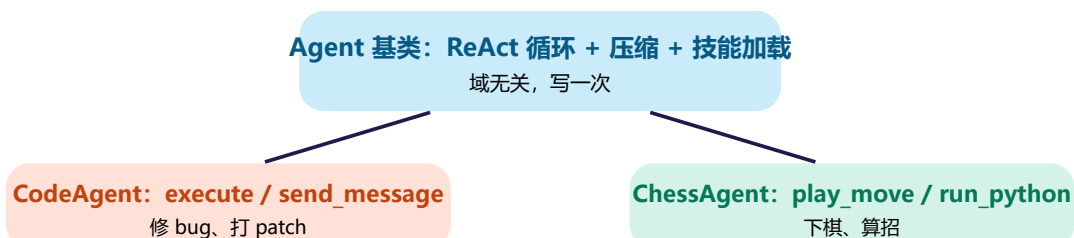
Q5

ReAct 循环长什么样？（作业 1 的核心 + 我的实现）

作业 1 要求在 **Agent** 基类实现域无关的 ReAct 循环，再让 **CodeAgent**（修软件 bug）和 **ChessAgent**（下棋）各自只提供自己的工具。消息序列随步骤生长：

```
[system] 常设指令 + <system_information>（环境信息块）+ 技能目录
[user]    任务描述
[assistant] 推理 + tool_calls: [{id:"call_1", name:"execute", arguments:...}]
[tool]    {"tool_call_id":"call_1", "content":"命令输出（超 1 万字符掐头留尾）"}
[assistant] 下一步推理 + tool_calls ... ← 每一步都把全部历史重新发给模型
```

我按作业 TODO 实现了全部离线部分，四个关键件：① **build_prompt** 只读渲染消息序列；② **run** 循环（超步数抛 `StepLimitError`）；③ **execute_tool_calls** 把坏 JSON / 未知工具变成“可恢复的观察”而不是崩溃；④ **load_skills** 解析 SKILL.md 的 YAML frontmatter。换域复用的秘密：



✅ 实测：实现前 **pytest** 7 failed (`NotImplementedError`)，实现后 **15 passed**（含 chess 三件套），4 个 modal 计费测试按课程设计排除。

Q6

观察爆炸怎么办：截断 + 上下文压缩

Agent 每步都重发全部历史，一次 `cat` 一个 2.6 万字符的文件就能把 prompt 撑爆。作业给出两层防御：

第一层：单条观察截断（harness 硬规则）

超过 10,000 字符的输出，**头尾各留 4900 字符，中间换成省略提示**——模型仍能看到文件开头结尾，并被告知可以“读更窄的范围”。这是确定性代码，不花 token。

第二层：上下文压缩（模型自己做工作记忆）

超阈值时，把**老步骤**喂给模型总结成一段**工作记忆**（保留目标、已跑命令、具体结果、失败路线、下一步），**最近一步完整保留**；system/task 消息永不压缩。

实测：26,000 字符观察进上下文后，压缩发生

step1 prompt
≈ 275 tok

step2 读大文件后
≈ 2816 tok

压缩

工作记忆替换
≈ 409 tok

↓ -85%

压缩 prompt 里同时给出任务原文与老步骤原始 JSON（保真），产出一段散文摘要；
最终 prompt 验证：长文件内容已不在、<working_memory> 摘要在、最近一步原样保留。

与本库 [[上下文预算与战略压缩]] 互补：那是“人给 agent 定预算策略”，这是“harness 自动做的工作记忆”

Q7

Skills 渐进披露怎么实现？

技能（Skill）是提前写好的“一类工作的操作手册”。问题：技能多了、每个又长，全塞进 system prompt 会吃光预算。作业采用 **Agent Skills 协议的渐进披露（progressive disclosure）**：

层	内容	何时进上下文
第 1 层：目录	每个技能一行 [name: description]（来自 SKILL.md 的 YAML frontmatter）	常驻 system prompt
第 2 层：全文	SKILL.md 正文（如 submit-task 的三步提交协议）	模型主动调 <code>invoke_skill(name)</code> 才加载

实测验证（demo 输出）

catalog names submit-task in prompt: **True**

skill body (patch.txt) hidden until invoked: **True** # 正文绝不提前泄漏

作业还埋了一个“负面测试”：技能不存在时，prompt 里**不许**出现 patch.txt 字样——防止模型凭幻觉自行发明提交协议。与本库 [[分层按需加载]]、[[AgentSkills技能包]] 完全同构：小入口常驻，大主体待命。

Q8

程序化工具调用强在哪？

普通工具调用一轮一个动作；**程序化工具调用 (programmatic tool calling)** 让模型写一段 Python，在沙箱里一次跑完"枚举候选招→模拟→评估→选最优"的整个程序——把"每步都要模型拍板"的循环，压缩成"一次代码执行"。作业 1 Part 3 的 `run_python` 就是它：模型写的代码经 base64 编码送进沙箱的 `sandbox_python.py` 执行（工具函数在那里可直接 import，棋局服务器就在一台本地），agent 进程从头碰不到这些代码。

一次一工具 (传统)

想→调→等→看→想→调...
每步都烧一次模型推理
多步棋类策略难表达

升级

程序化调用 (CodeAct 思路)

写一段代码：for move in legal: 模拟+评分
一次执行完成组合搜索
省推理次数，策略可编程

安全边界值得注意：代码**永远在沙箱执行**，返回值只是打印出的 JSON——模型代码与 agent 进程物理隔离，这正是 Q12 沙箱一课的伏笔。

Q9

SFT：把专家轨迹灌进模型 (讲 8)

监督微调 (SFT) = 拿大量"专家怎么干活"的完整轨迹 (任务→一系列推理+工具调用+观察→成功收尾)
当训练数据，让模型模仿。作业 1 的价值在此显现：你先亲手搭好 harness 跑出轨迹，作业 3 才有数据可训——**harness 产出的日志格式，就是 SFT 的训练样本格式** (课程读单 OLMo、StarCoder 即此类开放训练谱系)。SFT 便宜、稳定、教会"动作长什么样"；但它只能复制示范者水平，超越要靠 RL。

Q10

RL 训 Agent：奖励从哪来？（讲 9/11/12）

课程用整整四讲（SFT→RL 基础→高级算法→RL Systems）回答“怎么让 agent 比示范者更强”。核心是**可验证奖励（verifiable reward）**：

域	奖励信号	为什么适合 RL
Coding	测试是否通过	自动、客观、秒级判定（SWE-Bench 即此思路）
Chess	赢 / 输 / 规则合法	规则引擎就是免费裁判
Deep Research	答案可核对 / 引用真实	部分可验证，需设计

RL Systems 一讲（客座 Apurva Gandhi）讲工程真相：RL 训练的瓶颈常不在梯度，而在 **rollout——要让模型对成千上万个任务各自跑完整 agent 轨迹**，沙箱基建、生成吞吐、样本回收构成了真正的系统设计题。这解释了课程为何给作业配了 Modal 云沙箱与算力赞助商。

Q11

安全与框架课教什么？

Safety (7a / 9b)

沙箱与凭证管理：模型生成的代码不可信，执行必须隔离（作业里的 Modal 沙箱、Q8 的 base64 送代码都是实例）；**可观测性**：agent 生产环境要能回放每一步（作业强制保存轨迹 JSON 正是这个纪律的最小版）。

Frameworks (7b / 9a)

OpenHands（Neubig 亲授）与 **LangGraph**：手写 harness 之后再看框架，才知道框架帮你省了什么、藏了什么。本库 [[LangGraph与Agent编排]]、[[AgentHarness智能体挽具]] 可对照。

Q12

这门课给我什么？能力清单 + 跟学路线

我能用它干什么

① **系统自查**：拿课程大纲当"Agent 工程师知识地图"，逐项对照自己的知识库查漏（本库 agent 概念已覆盖①④板块大半，③训练板块是最大增量）；② **读 12 份 slides** 每份 1 小时，等于 12 次高质量综述；③ **白嫖作业链**：A1 离线测试零成本入门（本指南已打通），有 API key 可继续真跑；④ **照葫芦画瓢**：把作业 1 的 harness 骨架改造成自己的小 agent（换环境、换工具）；⑤ **前沿雷达**：客座讲者阵容就是 Agent 领域"谁在做真东西"的名单。

建议跟学路线（省时间版）

Lecture 01（总纲，已读）→ 03b Coding Agents → 04b/05a/05b（SFT+RL，知识增量最大）
→ 07a 沙箱安全 → 06a/06b RL 进阶 → 挑感兴趣领域补课
作业路线：A1 离线测试（已完成）→ A1 真跑（需 Modal+API）→ A2 Eval → A3 Training

⚠ 成本提示：作业真跑需 Modal 云沙箱 + LLM API（课程默认 DeepSeek-V4-Flash），均为计费服务；离线测试（make test / pytest 非 modal 标记）完全免费，本指南实验全部零成本完成。



动手实验：把作业 1 的离线部分全部打通（真实执行）

Step 1 • 克隆官方作业仓库并建环境

```
git clone https://github.com/cmu-agents/assignment-1 repo
cd repo && py -3.12 -m venv .venv
.venv/Scripts/python -m pip install httpx jinja2 openai python-dotenv pyyaml tqdm pytest modal sw
```

目的：拿到课程官方起手代码。**结果：**commit 67498d8 ("Update submission guidelines")。验证：`git log --oneline -1` 输出上述 hash。

Step 2 • 跑基线测试，看"官方留白"

```
PYTHONPATH=src .venv/Scripts/python -m pytest tests/test_agent.py -q --basetemp=../exercise/_pyte
```

结果： 7 failed ——全部 NotImplementedError，即作业留给学生实现的 TODO。**验证：**失败列表 = build_prompt / run / load_skills / execute_tool_calls 等。

Step 3 • 实现域无关 ReAct 循环 + 技能加载 + 上下文压缩

按 ASSIGNMENT.md 的 TODO 逐个填空（实现要点见 exercise/我的实现笔记.md）：消息状态与 build_prompt、run 主循环、坏输入变可恢复观察、SKILL.md frontmatter 解析、工作记忆压缩。再用同样套路补齐 Part 3 棋类三件套（三个 strict 工具 schema + 四个带 <chess_error> 错误通道的 helper + 分发器）。

Step 4 • 复跑全部离线测试

```
PYTHONPATH=src .venv/Scripts/python -m pytest tests/ -q --basetemp=../exercise/_pytest_tmp
```

✅ **预期结果（实测）：** 15 passed, 4 deselected——deselected 的是 modal 标记的计费沙箱测试，离线零成本全绿。

Step 5 • 脚本化 LLM 端到端演示 (零 API 费看 agent 活着)

```
.venv/Scripts/python ../exercise/demo_react_loop.py
```

✅ 预期结果 (实测摘要) : 5 步 ReAct 循环跑通; 26,000 字符长观察触发截断; 渐进披露双检查为 True (目录在、正文不泄漏) ; 压缩把 prompt 从 ≈ 2816 tokens 砍到 ≈ 409 (-85%) ; 轨迹落盘 demo_trajectory.json。

资源索引: 官网 cmu-agents.com · 作业 [cmu-agents/assignment-1](https://cmu-agents.com/assignment-1) · 推文来源

x.com/Xudong07452910/status/2097846768323133559

本地产出: repo (作业实现) · exercise/demo_react_loop.py + demo_trajectory.json + 我的实现笔记.md · vault 新

概念: [[Agent后训练SFT与RL]]、[[程序化工具调用]]

生成: 2026-10-03 · 拾遗任务 4ffdd734 · 学习方法: 疑问驱动 + 真机实验